TechBharat Cohort #2 Buildathon Use Cases

Use Case A: Chrome Extension for On-Screen Summarization

A.1 The problem

A knowledge worker spends the day inside a browser: long email threads, dense internal wikis, dashboards with forty metrics, sixty comment pull requests, regulatory PDFs, research papers. The information they need is almost always on screen already. The cost is the reading.

Today the workaround is manual: select text, copy, switch tabs, paste into a chatbot, ask for a summary, switch back, lose your place. That loop is slow enough that most people simply skim instead, and skimming is where details get missed.

Build a Chrome extension that removes the loop. One shortcut, and whatever is on screen is summarized in place, in a form the user can act on.

A.2 What you are building

A Manifest V3 Chrome extension that captures the content the user is looking at, sends it to a model with the right context, and returns a structured, trustworthy summary inside the page, without the user ever leaving the tab.

Must-have requirements

- Invocation from a toolbar icon and a keyboard shortcut, on any http or https page.
- Three capture modes: the whole page, the current text selection, and a user drawn region of the screen.
- Content extraction that survives real pages, not just clean article pages. Handle single page applications, lazy loaded content, iframes where permitted, and pages longer than the model context window.
- Structured output rather than one paragraph of prose. At minimum: a two line summary, key points, and any action items, decisions or numbers detected.
- Follow up questions against the captured context, so the user can ask "what did it say about pricing" without re-capturing.
- Streaming responses with a visible loading state. The user must see progress within two seconds of invoking.
- Copy to clipboard and export the summary as markdown.
- Graceful failure. If a page cannot be read, say so and explain why instead of returning an empty or hallucinated summary.

Stretch goals

- Visual understanding. Screenshot the region and summarize charts, tables and dashboards that have no useful text layer.
- Multilingual output. Summarize an English page into Hindi, Telugu, Tamil, Bengali or Marathi, and correctly summarize pages that are already in those languages.
- Cross tab synthesis. Select three open tabs and produce one comparative summary.
- Domain aware modes. A GitHub pull request, a Gmail thread and a Jira board each deserve a different summary shape. Detect the site and adapt.
- Searchable local history of past summaries, with the source URL and timestamp.
- Privacy mode. Detect and redact emails, phone numbers, PAN and Aadhaar style identifiers before anything leaves the browser, and offer a local or on-device model option.
- Inline highlighting. Map each summary point back to the part of the page it came from and scroll to it on click.

Constraints and non-functional requirements

- Manifest V3
- No API keys committed to the repository or shipped inside the extension bundle. Use a user supplied key stored in chrome.storage, or a thin backend proxy.
- Nothing is captured or transmitted without a deliberate user action. No background scraping.
- Request only the permissions you actually use, and be ready to justify each one to the judges.
- Target: first token visible within 3 seconds for a 2,000 word page on normal broadband.

Out of scope

- A full browser agent that clicks, fills forms or navigates on the user behalf. This track is about comprehension, not automation.
- Publishing to the Chrome Web Store. A loadable unpacked build is enough.

A.3 Success metrics

Judges will run your extension against a common test set of pages

Test Pages

1. https://paradigmit.ai/
2. https://www.thehindu.com/
3. https://www.eenadu.net/
4. https://docs.python.org/3.13/library/sched.html
5. https://play.grafana.org/d/000000110/business-metrics?orgId=1&from=now-24h&to=now&timezone=browser
6. https://github.com/anthropics/claude-agent-sdk-python/pull/1076
7. https://arxiv.org/pdf/1706.03762
8. https://www.seangoedecke.com/llms-reward-expertise/

Metric	Target	How it is measured
Page coverage	Usable summary on test pages	Judges run the test set
Faithfulness	Zero invented facts across 5 spot checked summaries	Judge compares summary claims to the page
Compression	90 percent or more length reduction with key points intact	Judge reads page and summary
Time to first token	Under 3 seconds at p50	Observed during the demo
Effort to invoke	Two clicks or one shortcut, no copy paste	Observed during the demo
Failure honesty	Blocked pages produce a clear error, never a fake summary	Judge tests one unreadable page

A.4 Suggested demo narrative

Open a 4,000 word internal policy page you have never seen. Summarize it. Ask two follow up questions. Then switch to a dashboard, region select a chart, and get the trend explained. Then show what happens on a page your extension cannot read.

Use Case B: Agentic AI Meeting Assistant

B.1 The problem

Meetings generate two things worth keeping: decisions and commitments. Both are routinely lost. Someone volunteers to do something on a Tuesday, nobody writes it down, and three weeks later the work has not started and no one can remember who owned it.

Transcription tools have largely solved capture. A transcript, though, is just the meeting again in text form, and nobody reads it. The gap is between the transcript and the tracker. Something has to read what was said, work out who agreed to do what by when, and put it where the team actually works.

This track asks for an agent, not a summarizer. The difference is that it takes action in the real world, under human supervision.

B.2 What you are building

A system that ingests a meeting, produces a structured record of what was decided, extracts action items with real owners and real dates, and then creates those items in an actual tool after a human approves them.

Must-have requirements

- Ingest a transcript file (txt, vtt or srt) or an audio or video recording. If you take audio, you own the transcription step.
- Attribute statements to speakers, either from provided labels or through diarization.
- Produce a structured meeting record containing: an executive summary, the decisions made, open questions, risks or blockers raised, and the action items.
- Extract action items with an owner, a due date, a priority and a confidence score. Resolve relative dates such as "by next Friday" or "end of the quarter" into actual dates using the meeting date.
- Resolve owners to real identities. "Priya will take this" must map to a specific person or email, and must fail loudly when it cannot.
- At least one genuine side effect through a tool integration: create issues in Jira, Linear, Asana or GitHub, post a recap into Slack or Teams, create calendar follow ups, or send a recap email.
- A human in the loop review screen. Nothing is created, sent or posted until a person sees the exact payload and approves it. Individual items must be editable or rejectable before approval.
- Idempotency. Running the same meeting twice must not create duplicate tasks.
- An audit log showing every action the agent took, what it was based on, and who approved it.

Stretch goals

- Code switched speech. Indian standups routinely mix English with Hindi, Telugu or Tamil in a single sentence. Handling that well is a genuine differentiator.
- Cross meeting memory. For a recurring meeting, carry forward open items, mark ones that were completed, and flag commitments that have slipped twice.
- Automated follow up. Nudge an owner before a due date, and detect in the next meeting whether the item was actually addressed.
- Evidence links. Every action item points back to the exact transcript timestamp where it was agreed.
- Disagreement detection. Surface where the room did not actually reach consensus, instead of flattening it into a decision.
- Meeting health analytics: talk time distribution, decision count, proportion of time on agenda.

- Live mode that runs during the meeting and surfaces action items as they are committed to.

Constraints and non-functional requirements

- No unapproved side effects, ever. An agent that emails the wrong person fails the track regardless of everything else.
- Handle a 45 minute meeting end to end in under 5 minutes.
- Use test or sandbox workspaces for integrations. Do not connect to a live production tracker for the demo.
- Do not use recordings of real meetings containing confidential or personal information without consent.

B.3 Success metrics

A gold standard transcript with human labelled action items is released at kickoff. Judges will run your system against it and against one unseen transcript.

Metric	Target	How it is measured
Action item recall	80 percent or more of labelled items found	Against the gold transcript
Action item precision	75 percent or more, few invented tasks	Against the gold transcript
Owner accuracy	85 percent or more correctly attributed	Against the gold transcript
Date resolution	90 percent of relative dates resolved correctly	Spot check of five items
End to end latency	Under 3 minutes for a 45 minute meeting	Timed during the demo
Unapproved actions	Exactly zero	Audit log review
Duplicate suppression	Re-run creates no duplicates	Judge runs the same file twice

B.4 Suggested demo narrative

Drop in a messy 40 minute transcript the judges have not seen. Show the structured record. Show the review screen, edit one owner, reject one weak item, approve the rest. Show the tickets appearing in the tracker and the recap in Slack. Then re-run the same file and show that nothing duplicates.

Use Case C: AI Code Review Bot for GitHub Pull Requests

C.1 The problem

Code review is the biggest queue in most engineering teams. Pull requests sit for a day waiting on a reviewer, and when review does happen a large share of the comments are about naming, formatting and missing null checks, which is exactly the work a machine should be doing. Meanwhile the things that matter, a race condition or a query that will fall over at scale, slip through because the reviewer is tired and the diff is 900 lines.

The failure mode of AI review bots is well known: they generate forty low value comments per pull request, developers mute them within a week, and the tool dies. Signal to noise is the entire problem here, and it is what this track is really testing.

C.2 What you are building

A bot that installs on a GitHub repository, reviews pull requests automatically, and leaves a small number of high value inline comments that a senior engineer would agree with.

Must-have requirements

- Installable on a repository as a GitHub App, GitHub Action or webhook service. Judges must be able to point it at a test repository.
- Triggers automatically when a pull request is opened and when new commits are pushed to it.
- Posts inline comments anchored to specific lines in the diff. A single summary comment is not sufficient.
- Every finding is categorised as correctness, security, performance, maintainability or test coverage, and given a severity.
- A pull request level summary comment: what changed, the risk areas, and a reviewer checklist.
- Context beyond the diff. When a changed function is called elsewhere, the review should reflect that rather than reasoning about twenty isolated lines.
- Incremental re-review. New commits get reviewed without re-posting comments that already exist.
- A configuration file in the repository controlling ignored paths, severity threshold, maximum comments and review focus.
- A hard cap on comments per pull request, with findings ranked by severity so the cap keeps the important ones.

Stretch goals

- Committable fixes using the GitHub suggested changes format, so a reviewer can accept a fix in one click.
- Repository awareness. Index the codebase so the bot enforces the conventions of this project rather than generic best practice.
- Learning from feedback. Track which comments get resolved versus dismissed, and suppress categories the team keeps rejecting.
- Test gap detection with generated test cases for the uncovered path.
- Secret and credential detection in the diff, plus flagging of risky new dependencies.
- A confidence score per comment, with low confidence findings collapsed rather than posted loudly.
- A cost and latency report per pull request, since a bot nobody can afford to run is not a product.

Constraints and non-functional requirements

- Never merge, close, force push or modify branches. The bot comments, humans decide.
- Review posted within 3 minutes of a pull request being opened.

- Maximum 10 comments per pull request by default.
- Handle large diffs sensibly through chunking or prioritisation rather than truncating silently.
- Report token cost per review.

C.3 Success metrics

A benchmark repository with a set of pull requests is released at kickoff. Some contain deliberately injected bugs of known severity. Judges may also open unseen pull requests.

Metric	Target	How it is measured
False positive rate	Under 30 percent of comments	Judge marks each comment valid or not
Actionability	70 percent of comments a judge would act on	Judge review of a live pull request
Comment volume	10 or fewer per pull request	Observed on the benchmark
Latency	Review posted within 5 minutes	Timed on the live pull request
Incremental behaviour	No repeated comments after a new commit	Judge pushes a follow up commit
Cost per review	Reported and reasonable	Self reported, verified in logs

C.4 Suggested demo narrative

Let a judge open a pull request on your test repository live. Walk through the comments it produces and defend each one. Push a commit that fixes two issues and show the re-review behave correctly. Show the config file suppressing a category. Show your cost per review.

Frequently Asked Questions

Use Case A specifics

Does the extension need to be published to the Chrome Web Store?

No. A loadable unpacked build or a packed zip is sufficient.

Can we build for Firefox or Edge instead?

The brief is a Chrome extension. Cross browser support is a fine stretch goal, but Chrome must work.

Are we allowed to send page content to a hosted model API?

Yes, provided this is disclosed in your README and the user triggers it deliberately. Doing on-device redaction or offering a local model option is a stretch goal, not a requirement.

Use Case B specifics

Do we need to build our own speech to text?

No. Using a transcription API or an open model is expected. You can also accept transcripts directly and skip audio entirely, though handling audio scores better on technical depth.

Which tracker should we integrate with?

Any one of Jira, Linear, Asana, GitHub Issues, Slack, Teams, Google Calendar or email. One deep, reliable integration beats four shallow ones.

What counts as agentic here?

The system decides what actions are needed and executes them through tools, rather than only producing text for a human to copy. The human approval step does not make it less agentic. It makes it safe.

Can we auto-send without approval if the user opts in?

Not for the demo. Zero unapproved actions is a hard metric on this track.

Use Case C specifics

Does it have to be a GitHub App?

No. A GitHub Action or a webhook service is equally acceptable, as long as judges can install it on a test repository and it triggers automatically.

How many languages must the bot support?

One language, done well, is the requirement. The benchmark repository will tell you which languages appear. Multi language support is a stretch goal.

Is it better to post more findings or fewer?

Fewer. False positive rate and comment volume are both scored. A bot that posts four correct comments beats one that posts thirty containing the same four.

Can the bot approve or merge pull requests?

No. Merging, closing and force pushing are prohibited. The bot comments and a human decides.